

2303A510A1 - Bhanu

Batch-14

Assignment -9.4

Documentation generation: Automatic
Documentation and Code Comments

Task 1: Auto-Generating Function Documentation in a Shared Codebase

Prompt:

Given a Python script containing multiple functions without any docstrings.

Code:

```
1  import math
2  def calculate_area(radius):
3      return math.pi * radius * radius
4  def is_prime(number):
5      if number <= 1:
6          return False
7      for i in range(2, int(number ** 0.5) + 1):
8          if number % i == 0:
9              return False
10     return True
11 def fibonacci(n):
12     sequence = []
13     a, b = 0, 1
14     for _ in range(n):
15         sequence.append(a)
16         a, b = b, a + b
17     return sequence
18 def find_max(numbers):
19     if not numbers:
20         return None
21     maximum = numbers[0]
22     for num in numbers:
23         if num > maximum:
24             maximum = num
25     return maximum
26 def sort_numbers(numbers):
27     for i in range(len(numbers)):
28         for j in range(0, len(numbers) - i - 1):
29             if numbers[j] > numbers[j + 1]:
30                 numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
31     return numbers
```

Output:

```
Ai.Ass.9.4.py • import math Untitled-1 •
Ai.Ass.9.4.py > ...
1  import math
2  def calculate_area(radius):
3      """Calculate the area of a circle.
4
5      Computes the area of a circle given its radius using the formula
6      A =  $\pi$  * r2.
7
8      Args:
9      |   radius (float): The radius of the circle.
10
11     Returns:
12     |   float: The area of the circle.
13
14     Example:
15     |   >>> area = calculate_area(5)
16     |   >>> print(round(area, 2))
17     |   78.54
18     """
19     return math.pi * radius * radius
20
21 def fibonacci(n):
22     """Generate the first n numbers in the Fibonacci sequence.
23
24     Creates a list containing the first n numbers of the Fibonacci sequence,
25     where each number is the sum of the two preceding ones, starting from 0 and 1.
26
27     Args:
28     |   n (int): The number of Fibonacci sequence elements to generate.
29
30     Returns:
31     |   list: A list containing the first n Fibonacci numbers.
32
33     Example:
34     |   >>> fibonacci(8)
35     |   [0, 1, 1, 2, 3, 5, 8, 13]
36     |   >>> fibonacci(5)
37     |   [0, 1, 1, 2, 3]
38     """
39     sequence = []
40     a, b = 0, 1
41     for _ in range(n):
42         sequence.append(a)
43         a, b = b, a + b
44     return sequence
45
```

```
Ai.Ass.9.4.py • import math Untitled-1 •
Ai.Ass.9.4.py > ...
45 |
46 | def find_max(numbers):
47 |     """Find the maximum value in a list of numbers.
48 |
49 |     Iterates through the list to identify and return the largest number.
50 |     Returns None if the list is empty.
51 |
52 |     Args:
53 |     |   numbers (list): A list of numeric values (int or float).
54 |
55 |     Returns:
56 |     |   int/float or None: The maximum value in the list, or None if the list is empty.
57 |
58 |     Example:
59 |     |   >>> find_max([3, 9, 2, 15, 7])
60 |     |   15
61 |     |   >>> find_max([-5, -2, -10])
62 |     |   -2
63 |     |   >>> find_max([])
64 |     |   """
65 |     if not numbers:
66 |         return None
67 |     maximum = numbers[0]
68 |     for num in numbers:
69 |         if num > maximum:
70 |             maximum = num
71 |     return maximum
```

Task-2: Enhancing Readability through AI-Generated Inline Comments

Prompt:

Automatically insert inline comments only for complex or non-obvious logic avoid commenting on trivial or self-explanatory syntax

Code:

```
Ai.Ass.9.4.py • 9.4.task.2.py •
9.4.task.2.py > ...
1  def fibonacci(n):
2      if n <= 1:
3          return n
4      return fibonacci(n - 1) + fibonacci(n - 2)
5
6
7  def binary_search(arr, target):
8      left, right = 0, len(arr) - 1
9      while left <= right:
10         mid = (left + right) // 2
11         if arr[mid] == target:
12             return mid
13         elif arr[mid] < target:
14             left = mid + 1
15         else:
16             right = mid - 1
17     return -1
```

Output:

```
Ai.Ass.9.4.py • 9.4.task.2.py X
9.4.task.2.py > ...
1  def fibonacci(n):
2      # Base case: handles both n=0 (returns 0) and n=1 (returns 1)
3      if n <= 1:
4          return n
5      # Combine the sum of the two previous Fibonacci numbers recursively
6      return fibonacci(n - 1) + fibonacci(n - 2)
7
8
9  def binary_search(arr, target):
10     left, right = 0, len(arr) - 1
11     while left <= right:
12         # Integer division avoids overflow; calculates midpoint between left and
13         mid = (left + right) // 2
14         if arr[mid] == target:
15             return mid
16         # If target is greater, search the right half by moving left boundary
17         elif arr[mid] < target:
18             left = mid + 1
19         # If target is smaller, search the left half by moving right boundary
20         else:
21             right = mid - 1
22     # Target not found in array
23     return -1
```

Task-3: Generating Module-Level Documentation for a python package.

Prompt:

Generate a small Python module with no comments

Code:

```
1  import math
2  def calculate_area(radius):
3      return math.pi * radius * radius
4  def calculate_perimeter(radius):
5      return 2 * math.pi * radius
6  def is_even(number):
7      return number % 2 == 0
8  def factorial(n):
9      if n <= 1:
10         return 1
11     return n * factorial(n - 1)
12 def find_max(values):
13     if not values:
14         return None
15     maximum = values[0]
16     for v in values:
17         if v > maximum:
18             maximum = v
19     return maximum
```

Output:

```
1  """Utility math helpers.
2  |
3  Purpose:
4  |   Small collection of mathematical helper functions for area/perimeter
5  |   calculations, simple numeric tests, and common sequence/aggregation
6  |   utilities.
7  |
8  Required libraries / dependencies:
9  |   - math (standard library)
10 |
11 Key functions:
12 |   - `calculate_area(radius)`: Return area of a circle for given radius.
13 |   - `calculate_perimeter(radius)`: Return circumference of a circle.
14 |   - `is_even(number)`: Return True if `number` is even.
15 |   - `factorial(n)`: Compute factorial of `n` (recursive).
16 |   - `find_max(values)`: Return the maximum value from an iterable or None
17 |   if empty.
18 |
19 Example:
20 |   >>> from 9.4.task.3 import calculate_area, factorial, find_max
21 |   >>> calculate_area(3)
22 |   28.274333882308138
23 |   >>> factorial(5)
24 |   120
25 |   >>> find_max([1, 7, 3])
26 |   7
27 |   """
```

Task - 4: Converting Developer Comments into Structured Docstrings.

Prompt:

Generate a Python code along with long explanatory comments inside functions instead of proper docstrings.

Code:

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n-1)
```

Output:

```
def factorial(n):  
    # This function calculates the factorial of a given number n.  
    # The factorial of a non-negative integer n is the product of all positive integers less than or equal to n.  
    # For example, factorial(5) would be 5 * 4 * 3 * 2 * 1 = 120.  
    # Base case: if n is 0, the factorial is defined to be 1.  
    if n == 0:  
        return 1  
    # Recursive case: if n is greater than 0, we multiply n by the factorial of (n-1).  
    # This is because the factorial of n can be expressed as n * (n-1)!.  
    return n * factorial(n-1)
```

Task-5: Building a Mini Automatic Documentation Generator

Prompt: Generate a small Python utility that reads a given .py file automatically detects, Functions, Classes. Inserts placeholder Google-style docstrings for each detected function or class.

Output:

```
import re
def generate_doc(name):
    return f'''
    {name} description.
    Args:
        TODO: Describe parameters.
    Returns:
        TODO: Describe return value.
    '''
...
def process_file(filename):
    with open(filename, "r") as f:
        lines = f.readlines()
    new_lines = []
    for line in lines:
        new_lines.append(line)
        match = re.match(r"def (\w+)\(.*\):", line)
        if match:
            new_lines.append(generate_doc(match.group(1)))
    with open("output.py", "w") as f:
        f.writelines(new_lines)
```